

X. Conclusion:

In this practical, we implemented Linear, Logistic, and Ridge Regression models using suitable datasets. We trained the models, made predictions, and evaluated their performance using metrics like MSE, R^2 score, and Accuracy. This helped us understand how regression techniques are used to predict outcomes in real-world applications.

XI. Practical Related Questions

1. What is regression in machine learning?

Regression is a **supervised learning technique** used to predict **continuous numeric values** based on input features. It models the relationship between dependent (target) and independent (input) variables.

2. Difference between regression and classification

Regression	Classification
Predicts continuous values	Predicts categorical values
Example: Predict house price	Example: Predict spam or not spam
Output: Numeric	Output: Class label

3. Real-life examples of regression

- House price prediction
 - Salary prediction based on experience
 - Sales forecasting
 - Temperature prediction
 - Stock market price prediction
-

4. What type of data is used in regression problems?

Regression uses:

- Numerical/continuous dependent variable
 - Independent variables can be numerical or categorical (after encoding)
-

5. Which regression algorithm did you use and why?

We used:

- **Linear Regression** – Simple and effective for linear relationships
 - **Logistic Regression** – Used for classification problems
 - **Ridge Regression** – Prevents overfitting using regularization
-

6. What does `fit()` and `predict()` do in scikit-learn?

- `fit()` → Trains the model using training data
 - `predict()` → Predicts output using trained model
-

7. What is Mean Squared Error (MSE)?

MSE measures the average squared difference between actual and predicted values.

$$MSE = \frac{1}{n} \sum (y_{\text{actual}} - y_{\text{predicted}})^2$$

Lower MSE = Better model performance.

XII Exercise:

1. Develop a program to demonstrate the working of Linear Regression. Use Boston Housing Dataset for Linear Regression.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
data = pd.read_csv("boston.csv")

# Correct feature names
important_features = ["rooms", "lstat", "ptratio", "crim"] # "rooms" instead of "rm"
X = data[important_features] # Input features
y = data["mv"] # Target variable ("mv" instead of "medv")

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create and train model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate model
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nModel Performance:")
print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
print("R2 Score:", r2)

# Compare actual vs predicted
comparison = pd.DataFrame({"Actual": y_test, "Predicted": y_pred})
print("\nActual vs Predicted:\n", comparison.head())
```

2. Write a program to implement the Linear Regression by using the suitable dataset.

```
# Linear Regression Example with Visualization
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Dataset
data = {
    "Hours": [1, 2, 3, 4, 5, 6, 7, 8],
    "Marks": [10, 20, 25, 40, 45, 50, 60, 65]
}
df = pd.DataFrame(data)

# Features and target
X = df[["Hours"]]
y = df["Marks"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Evaluate
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print("Linear Regression Performance:")
print("MSE:", mse, "RMSE:", rmse, "R2:", r2)

# Visualization
plt.scatter(X, y, color='blue', label='Actual')
plt.plot(X, model.predict(X), color='red', label='Predicted')
plt.title("Linear Regression: Hours vs Marks")
plt.xlabel("Hours")
plt.ylabel("Marks")
plt.legend()
plt.show()
```

3. Write a program to implement the logistic regression by using the suitable dataset.

```
# Logistic Regression Example with Visualization
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# Dataset
data = {
    "Hours": [1, 2, 3, 4, 5, 6, 7, 8],
    "Pass": [0, 0, 0, 1, 1, 1, 1, 1]
}
df = pd.DataFrame(data)

X = df[["Hours"]]
y = df["Pass"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Train model
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Evaluate
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
print("Logistic Regression Performance:")
print("Accuracy:", accuracy)
print("Confusion Matrix:\n", cm)

# Visualization
import numpy as np
plt.scatter(X, y, color='blue', label='Actual')
plt.plot(X, model.predict_proba(X)[:,1], color='red', label='Probability')
plt.title("Logistic Regression: Hours vs Pass Probability")
plt.xlabel("Hours")
plt.ylabel("Pass Probability")
plt.legend()
plt.show()
```

4. Write a program to implement the Ridge Regression by using the suitable dataset.

```
# Ridge Regression Example with Visualization
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Dataset
data = {
    "Hours": [1, 2, 3, 4, 5, 6, 7, 8],
    "Marks": [10, 20, 25, 40, 45, 50, 60, 65]
}
df = pd.DataFrame(data)

X = df[["Hours"]]
y = df["Marks"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Train model
model = Ridge(alpha=1.0)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Evaluate
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print("Ridge Regression Performance:")
print("MSE:", mse, "RMSE:", rmse, "R2:", r2)

# Visualization
plt.scatter(X, y, color='blue', label='Actual')
plt.plot(X, model.predict(X), color='green', label='Predicted')
plt.title("Ridge Regression: Hours vs Marks")
plt.xlabel("Hours")
plt.ylabel("Marks")
plt.legend()
plt.show()
```

5. Evaluate model performance using metrics.

For Regression:

- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R² Score

For Classification:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-score

Regression Evaluation Metrics

These metrics are used when the output is **continuous (numeric values)**.

Mean Squared Error (MSE)

MSE measures the **average of squared differences** between actual and predicted values. It tells how far predictions are from real values.

□ Smaller MSE = Better model

♦ Formula:

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

Where:

- y = Actual value
- $\hat{y}$ = Predicted value
- n = Number of data points

📄 Program Code:

```
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(y_test, y_pred)
print("MSE:", mse)
```

Root Mean Squared Error (RMSE)

RMSE is the **square root of MSE**.

It shows error in the **same unit as the target value**.

□ Smaller RMSE = Better model

◆ Formula:

$$RMSE = \sqrt{MSE}$$

📄 Program Code:

```
import numpy as np
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

print("RMSE:", rmse)
```

R² Score (Coefficient of Determination)

R² shows how well the model explains the data.

It tells the percentage of variation explained by the model.

□ Value range:

- 1 → Perfect model
- 0 → Poor model
- Negative → Very poor model

◆ Formula:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Where:

- SS_{res} = Sum of squared errors
- SS_{tot} = Total variation

📄 Program Code:

```
from sklearn.metrics import r2_score

r2 = r2_score(y_test, y_pred)
print("R2 Score:", r2)
```